

포트폴리오

항목	내용
Email	c0z0c@naver.com
Tel	010-434*-****
GitHub	https://github.com/c0z0c
PyPI	https://pypi.org/user/c0z0c/

엔지니어 소개

22년간 고신뢰성이 요구되는 시스템을 만들어왔습니다.

철도 방송, 한전 전력 계량, IPTV - 오작동이 곧 사고나 서비스 중단으로 이어지는 현장에서, 신뢰성과 보안을 설계의 최우선 기준으로 삼아왔습니다. 그 위에 LLM·n8n·MCP 기반 AI 자동화 역량을 더해, 지금은 시스템 엔지니어의 정밀함으로 AI 솔루션을 구축하고 있습니다.

핵심 강점 3가지

- 고신뢰성 설계: Mission-Critical 환경(철도·전력)에서의 이중화·무결성 검증·이상 감지 설계 경험
- 보안·암호화 자체 구현: KCMVP ARIA/AES-256, DTLS 시리얼 암호화를 논문 분석 후 C++로 독자 구현
- 문제를 특허로: 현장 문제를 기술로 해결하고 다수 특허로 권리화

AI 종목 추천

1. 개요 (Overview)

1.1. 프로젝트 소개

AutoStock은 한국투자증권(KIS) Open API를 기반으로, 데이터 수집부터 종목 추천, 향후 자동 매수/매도까지 이어지는 개인 투자 자동화 파이프라인이다. 여러 서브 프로젝트가 한 저장소에 공존하는 모노레포(monorepo) 구조이며, 각 서버는 역할별로 분리되어 서로 REST/도구 호출로 연동된다.

1.2. 실행 결과 미리보기

아래는 파이프라인이 실제로 매일 저녁 생성해 Notion에 자동 업로드하는 보고서 화면이다. 코스피/코스닥 지수 추이와 시장 분위기 요약, 종목별 최근 120일 추이 차트, 추천 근거까지 사람이 작성한 것과 같은 형태로 매일 자동 발행된다.

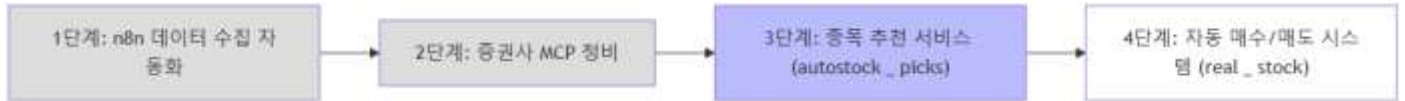


1.3. 개발 현황

전체 진행률은 약 65%다.

- **완료:** 증권사 MCP 톨 정비, 추천 종목 서비스(`autostock_picks`) - 질의응답형 종목 추천·비동기 폴링·보고서 생성에 더해, 재시도 루프(추천평가판단)와 시장 레짐 연동 매수가 산출까지 동작. n8n을 통한 추천 결과의 Notion/Telegram 자동 전달까지 연결
- **진행 예정:** 자동 매수/매도 시스템(`real_stock`) - 실시간 모니터링/주문 실행까지 확장

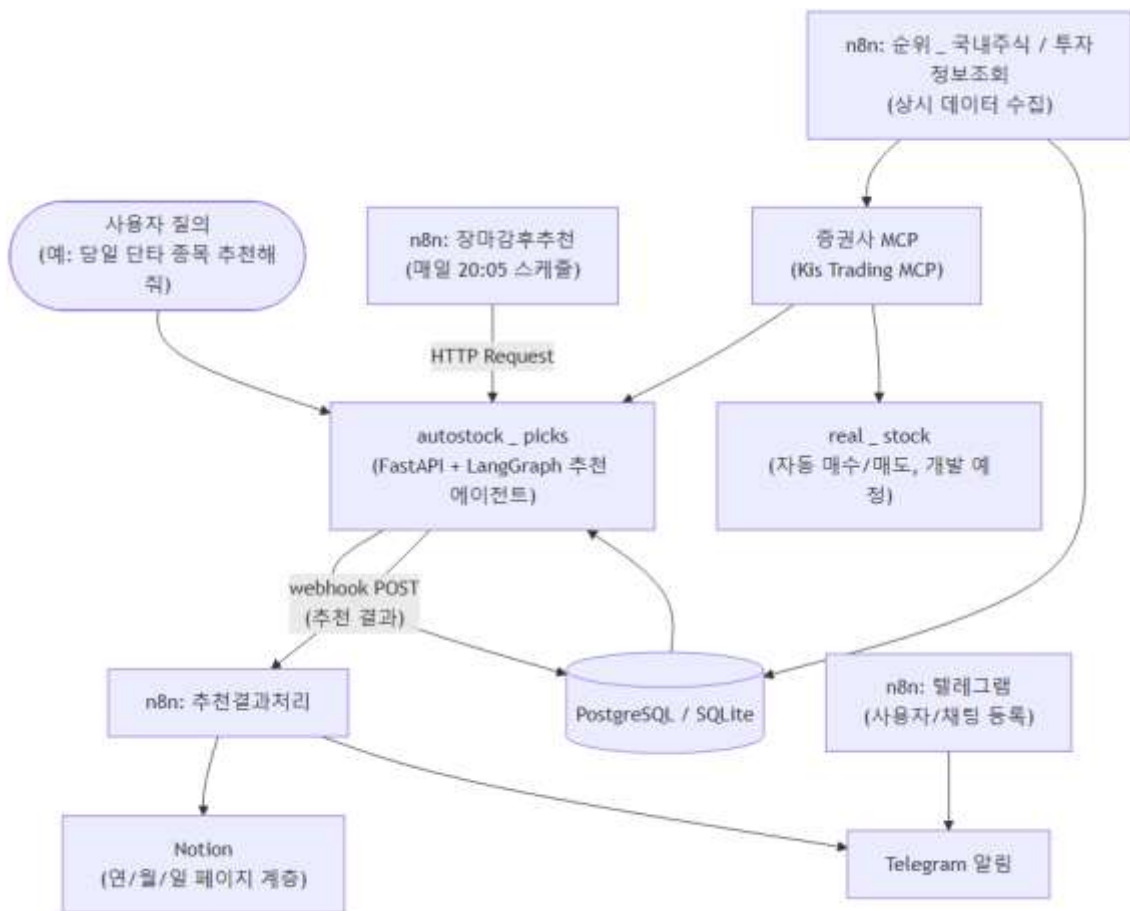
1.4. 로드맵



n8n 기반 데이터 수집부터 단계적으로 개발을 진행했다. 현재는 종목 추천 서비스가 동작 중이며, 다음 단계로 자동 매수/매도 시스템 개발이 예정되어 있다.

2. 시스템 아키텍처 (System Architecture)

2.1. 전체 구성도



2.2. 서버별 역할

2.2.1. 증권사 MCP (MCP Server)

한국투자증권 Open API를 도구(tool) 형태로 감싼 벤더(vendor) MCP 서버(open-trading-api/MCP/Kis Trading MCP)를 기반으로 사용한다. 원본 저장소와 지속적으로 패치/머지될 수 있어 폴더 구조는 임의로 바꾸지 않고, 다음 두 가지 방향으로 튜닝을 진행했다.

- **도구 명칭 표준화:** 164개 `api_type`을 `get_(조회)` / `order_(주문)` / `amend_(정정취소)` 그래머로 통일해 AI가 도구 목적을 더 쉽게 판단하도록 정리(`tasks/01_naming`, `tasks/02_naming`).
- **설명 간소화:** 8개 도구의 `<api_types>` 설명을 category 기준으로 그룹핑하고 불필요한 수식어를 제거해, LLM의 tool selection 정확도를 높였다.

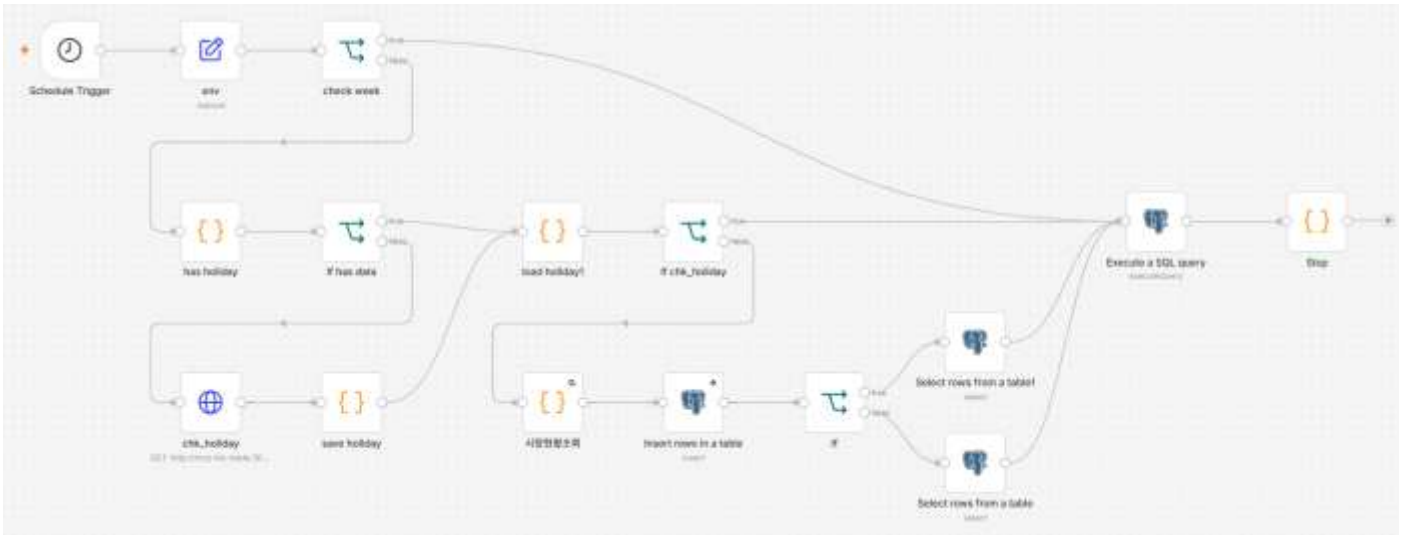
2.2.2. n8n (데이터 자동화 + 결과 전달)

시세/지수 데이터 수집부터 추천 결과 전달까지, 5개 워크플로로 나눠 자동화한다. 모든 스케줄 워크플로는 공통 휴장일 체크(`chk_holiday` 서브플로)를 거쳐, 휴장일이면 이후 단계를 건너뛴다.

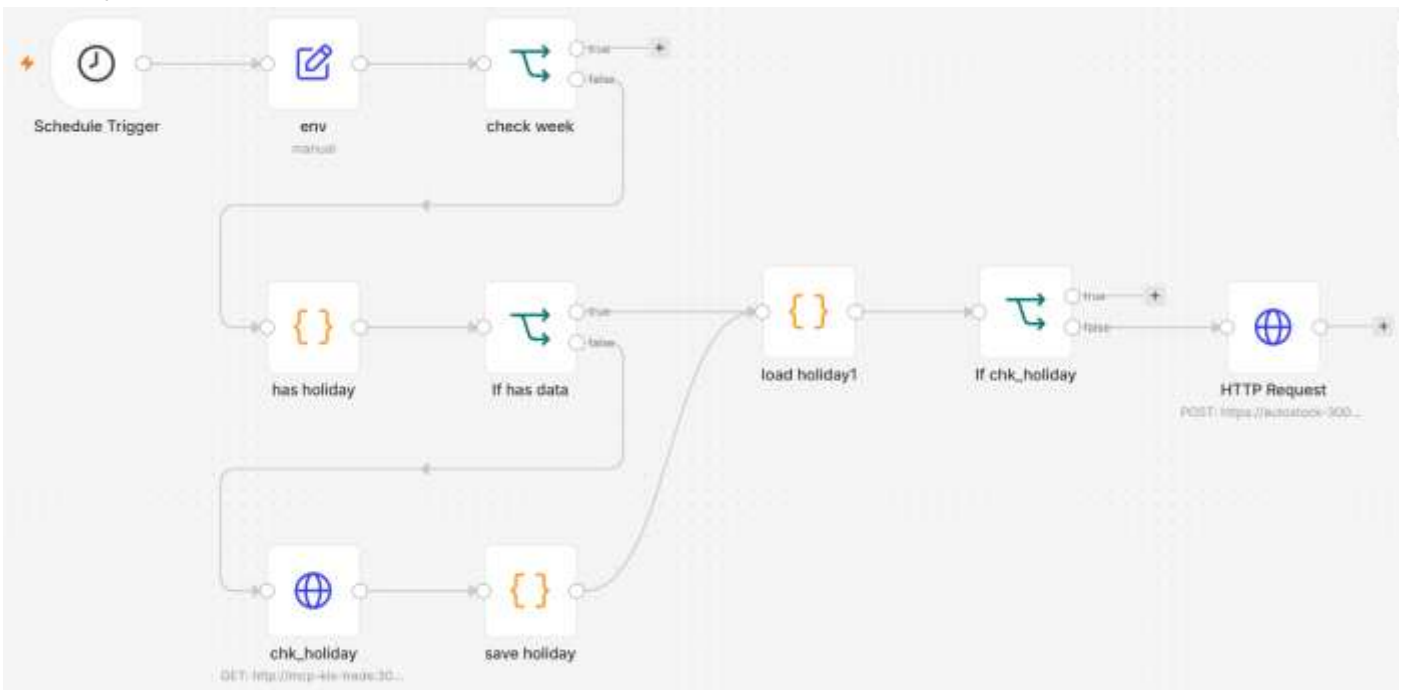
- 순위_국내주식: 거래량 순위(get_volume_rank_all)·등락률 순위(get_fluctuation_rank_all)를 조회해 DB에 적재
- 투자정보조회: 국내 주가 지수(코스피/코스닥) 등락률 현황을 주기적으로 조회해 DB에 적재
- 장마감후추천: 매일 20:05 autostock_picks에 추천을 HTTP Request로 요청해 파이프라인을 기동
- 추천결과처리: autostock_picks가 완료 시 webhook으로 보낸 추천 결과를 받아, Notion에 연/월/일 페이지 계층을 자동 생성·탐색하며 보고서를 업로드하고 Telegram으로 알림을 전송
- 텔레그램: Telegram 봇과 대화를 시작한 사용자/채팅방을 DB에 등록해, 추천결과처리가 알림을 보낼 대상을 관리

다음은 실제 동작 중인 n8n 워크플로 화면이다.

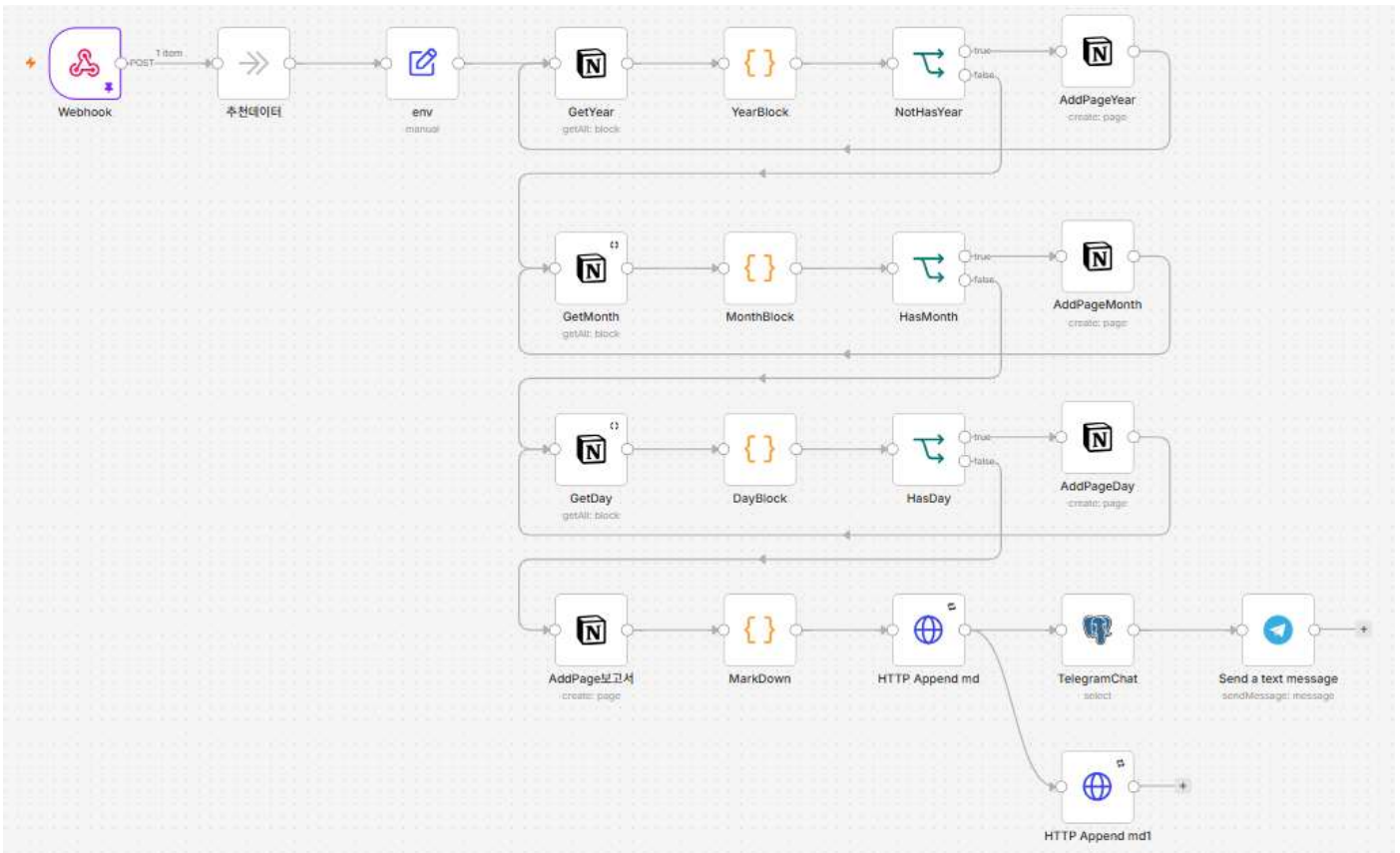
투자정보조회 – Schedule Trigger로 주기 실행하고, chk_holiday 서브플로로 휴장일이면 이후 단계를 건너뛴다. 정상 영업일이면 시장현황조회로 지수 등락률을 조회해 PostgreSQL에 insert/select한다.



장마감후추천 – 매일 20:05 Schedule Trigger로 기동하며, 동일한 휴장일 체크를 거친 뒤 autostock_picks에 HTTP Request(POST)로 추천을 요청해 파이프라인을 트리거한다.

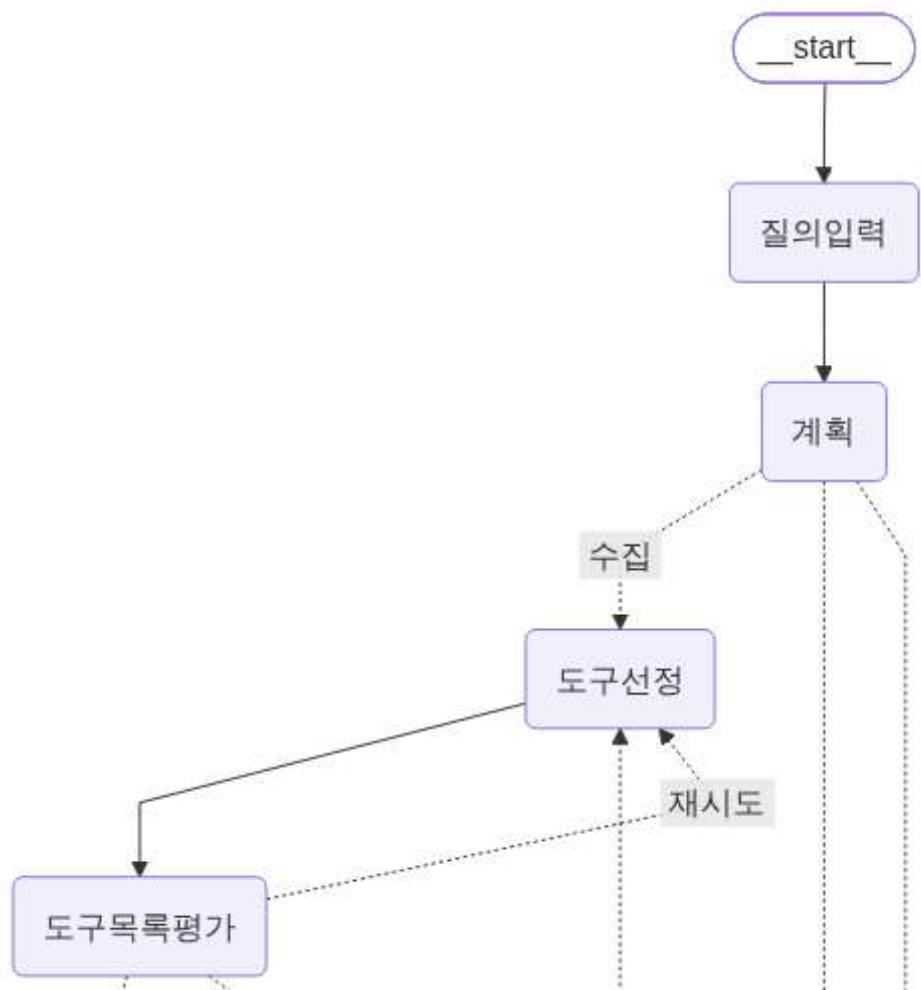


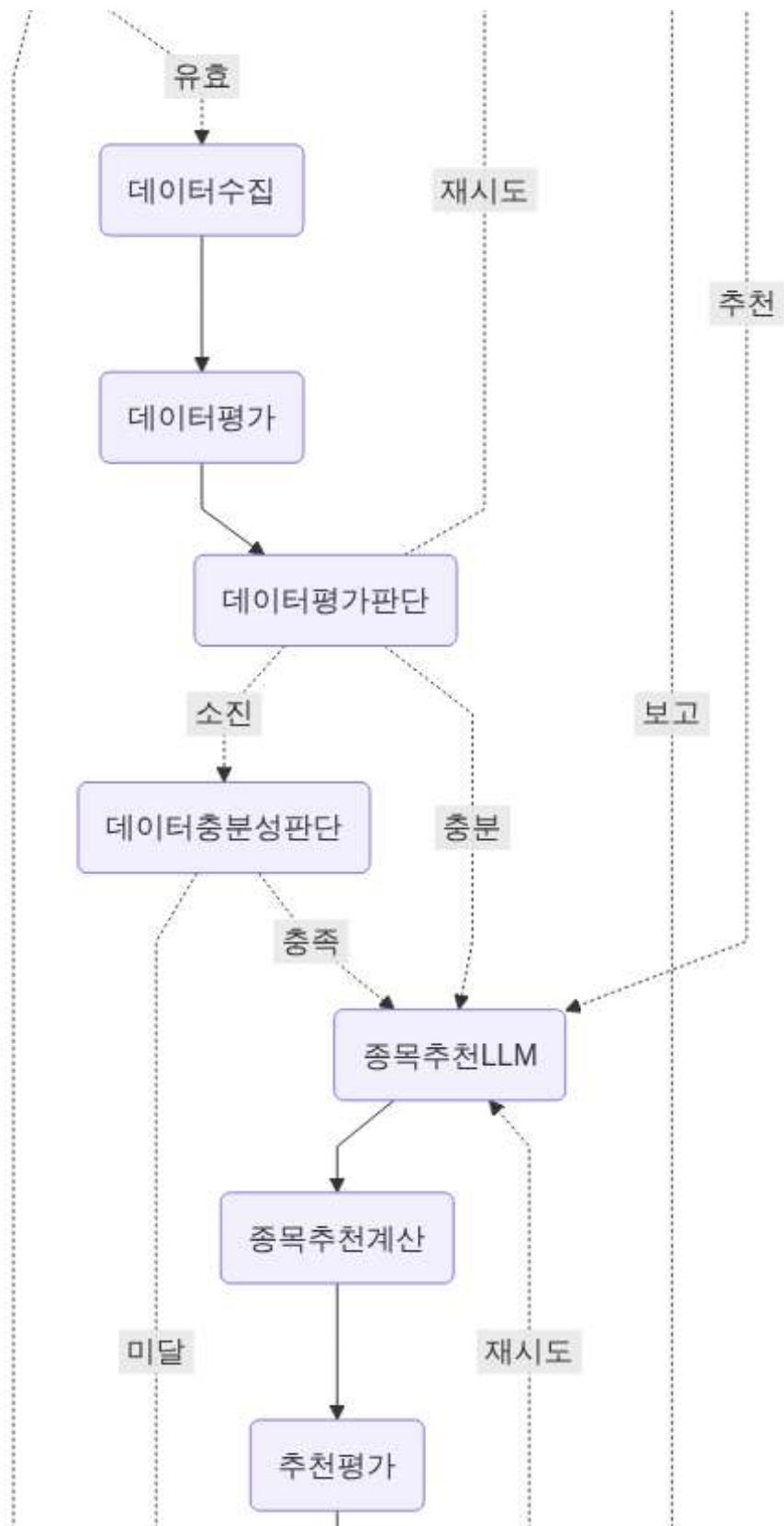
추천결과처리 – autostock_picks가 완료 시 보내는 Webhook을 받아, 연/월/일 단위로 Notion 페이지 존재 여부를 확인하며 없으면 생성하고, 보고서를 Markdown으로 업로드한 뒤 Telegram으로 알림을 전송한다.

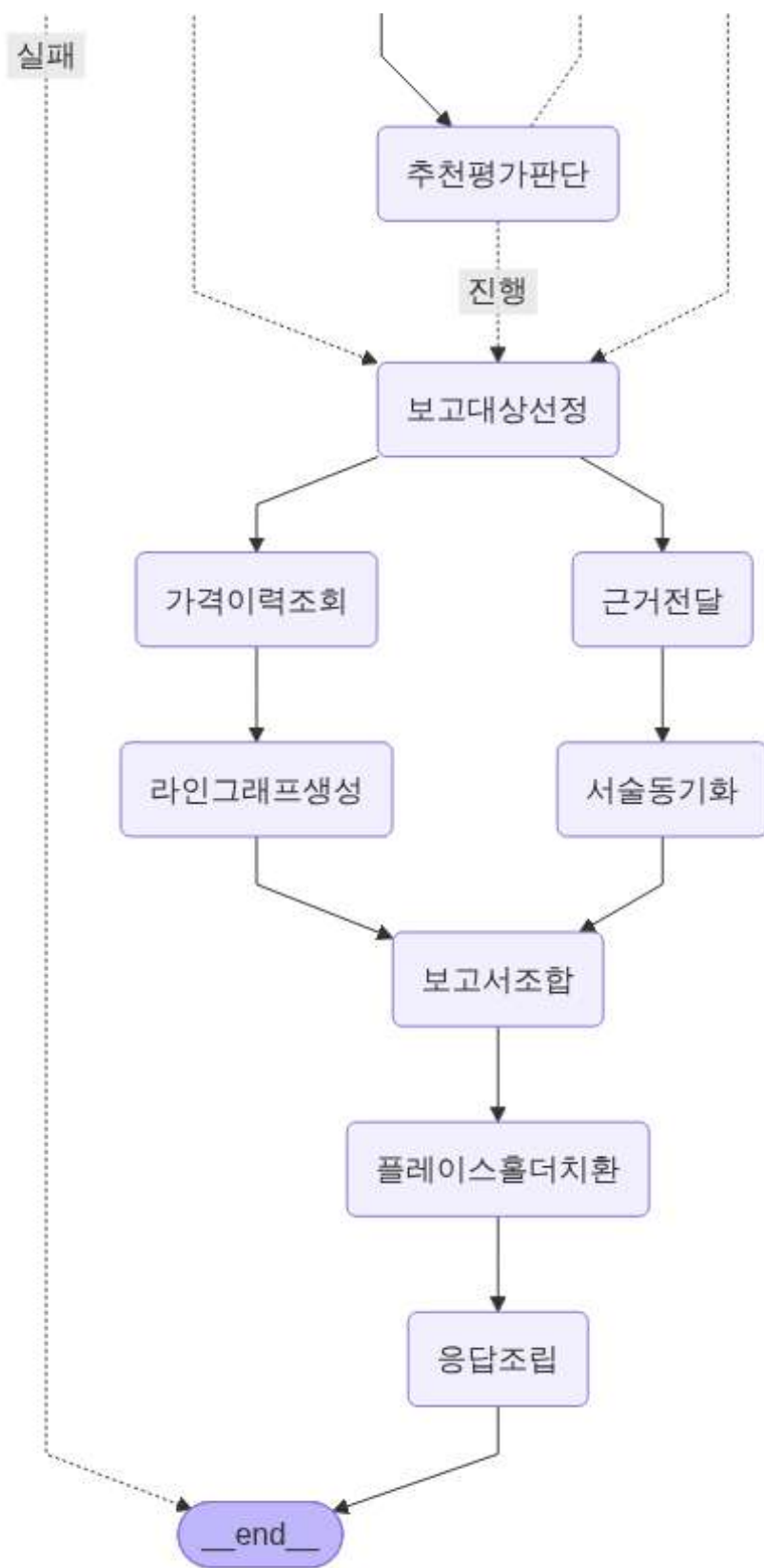


2.2.3. autostock_picks (추천 서비스)

증권사 MCP 도구와 n8n이 축적한 데이터를 근거로, 자연어 질의(예: "당일 단타 종목 추천해줘")에 답하는 LangGraph 기반 에이전트다. 개발 전 아래와 같이 전체 플로우를 mermaid로 먼저 설계하고, 이를 LangGraph로 구현해 검토한 뒤 코드를 작성하는 순서를 따랐다.







주요 흐름: 질의입력 → 계획 → 도구선정/도구목록평가 → 데이터수집/평가 → 데이터충분성판단 → 종목추천LLM/종목추천계산 → 추천평가/추천평가판단 → 보고대상선정 → 가격이력조회·근거전달 → 라인그래프생성·서술동기화 → 보고서조합 → 응답조립. 데이터 부족이나 도구 실패 시 재시도/대체 경로(dotted edge)로 순환하도록 설계했다.

추천평가는 규칙 기반 점수(추천 개수 충족도·근거 충실도·unresolved 비율·개별 점수 평균)에 장중/장마감 후 서로 다른 가중치를 적용하고, 필요 시 LLM 점수를 보조로 혼합하는 하이브리드 방식이다. 추천평가판단은 이 점수를 임계값과 비교해 미흡하면 종목추천LLM으로 재시도시키되, 추천 전량이 E0 정량 필터(등락률/유동성) 미통과 상태면 재시도해도 raw_data가 그대로라 개선이 불가능하다고 보고 재시도 없이 다음 단계로 넘긴다(불필요한 LLM 재호출로 인한 토큰 낭비 방지). 매수 기준가는 평균일중변동폭에 되돌림배수를 곱해 산출하는데, 이 배수는 당일 코스피/코스닥 평균 등락률로 판별한 시장 레짐(상승장/그 외)에 따라 결정적으로 달라진다(docs/종목추천시나리오.md 6단계).

2.2.4. real_stock (자동 매수/매도, 개발 예정)

특정 종목의 분봉을 REST로 채운 뒤 실시간에는 웹소켓으로 전환해 모니터링하고, 그리드 매수·trailing stop 매도 같은 조건에 따라 주문을 실행하는 FastAPI 백엔드를 설계 중이다. 현재는 API/서비스/DB/대시보드 인터페이스만 스캐폴딩되어 더미 데이터로 검증된 단계이며, 매매 로직 구현은 다음 개발 단계다.

2.2.5. 데이터베이스

PostgreSQL과 SQLite를 함께 사용한다. 서비스 간 실시간성이 필요한 데이터와 로컬 검증용 데이터를 구분해 저장한다.

3. 핵심 기능 (Core Features)

3.1. 증권사 MCP 톨 튜닝

AI(LLM)가 도구를 더 정확히 선택하도록 명칭과 설명을 정비했다. 164개 api_type 네이밍 통일과 8개 도구의 category 그룹핑을 완료했다(2.2.1 참고).

3.2. 상시 데이터 수집 (n8n)

n8n을 통해 국내 주식 거래량/등락률 순위, 지수 등락률 현황을 주기적으로 수집해 DB에 축적한다(2.2.2 참고). autostock_picks가 이 데이터를 참고해 추천을 산출한다.

3.3. LangGraph 기반 추천 에이전트

MCP 도구 실연동, LLM 벤더 확정(OpenAI), 정량 필터 미통과 후보의 점수 차감 처리, task_id 기반 비동기 실행과 진행률(현재 노드/퍼센트) 폴링 API에 더해, 규칙+LLM 하이브리드 추천 품질 평가와 시장 레짐 연동 매수가 산출, 무의미한 재시도를 스킵하는 재시도 루프까지 구현했다(tasks/05_autostock_picks).

3.4. 추천 보고서 생성

시장 현황 요약, 추천 종목별 최근 주가 상황, 매수 기준가/손절 라인, 차트 해설을 포함한 Markdown 보고서를 자동 생성한다. 아래는 실제 산출물 발췌다(script/output/reports/fastapi_260729_141400.md).

246710 티앤알바이오팜

왜 이 종목을 봐야 하나요 현재가 2535원, 전일대비 +180원, 전일대비율 7.64%로 단기 반등이 이어지고 있습니다. ... 다만 고가 2680원과 저가 1777원 사이 폭이 커서 변동성이 크고, 내일 단타는 2680원 돌파와 2355원 지지 여부를 같이 확인해야 합니다.

매수 기준가와 손절 라인 매수 기준가는 2436원, 손절 라인은 2246원입니다. 이 기준은 최근종가 2490원을 바탕으로, 평균일중변동폭 7.8%의 절반만큼 최근종가보다 낮게 잡아 매수 기준가를 정했고, 그 매수 기준가에서 다시 같은 흔들림 폭만큼 낮게 잡아 손절가를 둔 것입니다.

3.5. 추천 결과 자동 전달 (n8n 연계)

autostock_picks는 요청 옵션(options.webhook)으로 지정된 URL에 완료된 추천 결과를 JSON POST로 전달한다(동기 조회/비동기 task 완료 양쪽 공통). n8n의 추천결과처리 워크플로가 이를 수신해 Notion에 연/월/일 페이지 계층을 자동 생성·탐색하며 보고서를 업로드하고, 등록된 사용자에게 Telegram으로 알림을 보낸다. webhook 전송 실패는 파이프라인 결과 자체에 영향을 주지 않도록 예외를 흡수하고 경고 로그만 남긴다.

4. 개발 프로세스 및 품질 관리 (Development Process & QA)

4.1. 설계 우선 개발

새 로직을 개발할 때는 먼저 mermaid 그래프로 흐름을 그리고, LangGraph로 그 그래프를 검토한 뒤 실제 개발에 들어간다(docs/real_stock_design.md, docs/graph.png 참고).

4.2. 단계별 유닛 테스트

각 개발 단계마다 tests/에 유닛 테스트를 추가해 품질을 관리한다. 예: MCP 조회 함수 67종 구현 시 197건, autostock_picks 비동기 폴링 API 구현 시 11건의 테스트를 통과시켰다.

4.3. 성능 개선

과거 데이터는 값이 변하지 않는다는 특성을 활용해, 증권사 MCP의 과거 데이터 조회 구간에 캐시 로직을 추가해 조회 효율을 높이는 작업을 진행 중이다.

5. 기술 스택 (Tech Stack)

구분	기술
백엔드 프레임워크	FastAPI, Uvicorn, httpx
에이전트/LLM	LangGraph, LangChain(langchain-openai, langchain-mcp-adapters), OpenAI
MCP	fastmcp, mcp
데이터베이스	PostgreSQL, SQLite
데이터 처리	pandas
자동화	n8n
보고/알림	Notion API, Telegram Bot API
패키지/환경 관리	uv, .venv
테스트	pytest
인프라	Docker(MCP 서버 컨테이너 + autostock_picks 컨테이너, base 이미지 분리)

6. 향후 계획 (Roadmap)

- real_stock 매매 로직 구현: 그리드 매수, trailing stop 매도 조건 적용
- 실시간 웹소켓 모니터링과 자동 주문 실행 연동
- n8n 워크플로에 하드코딩된 API 키/토큰을 자격증명 관리 방식으로 분리

사내메신저 메시지 AI 자동 분석 및 업무 리소스 집계 시스템

기간: 2026.04 ~ 2026.06

역할: 기획 · 설계 · 개발 (단독) 스택: n8n · RAG · Claude AI (Haiku 4.5 / Sonnet 4.6) · Notion · 사내메신저 봇 · Python · Google Sheets

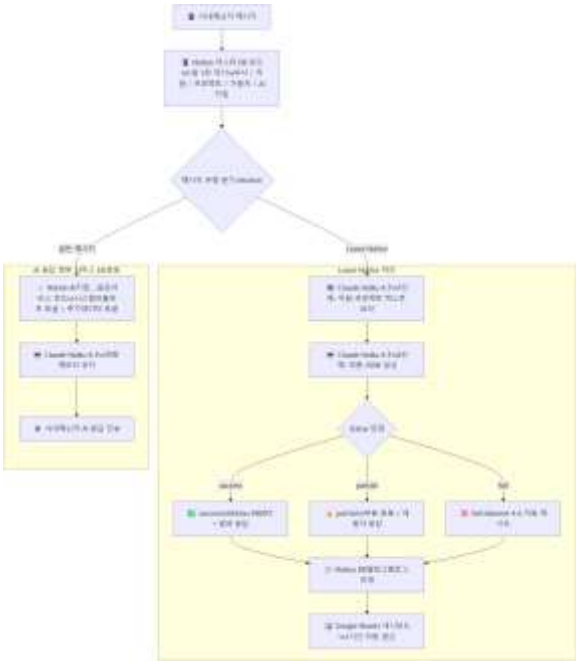
개요

팀원들이 사내메신저 채널에 자유 형식으로 입력하는 외근·출장·회의 메시지를 AI가 실시간 분석하여 Notion DB에 자동 적재하고, 주간·월간 업무 보고서를 자동 생성하는 NoCode 자동화 시스템.

문제 → 해결

Before	After
비정형 텍스트 메시지로 공유 → 수작업 정리	AI가 자동 파싱 → Notion DB 즉시 적재
프로젝트별 인력 투입 현황 파악 불가	실시간 대시보드로 현황 조회
주간/월간 보고서 수작업 작성	정해진 시각에 자동 생성

시스템 구조



주요 설계 포인트

AI 2단계 추론 단일 호출 시 출력 토큰 잘림 문제 → 1단계(텍스트 요약) + 2단계(JSON 생성)로 분리
Haiku → Sonnet 자동 Fallback 분석 fail 또는 JSON 파싱 오류 시 Sonnet 4.6으로 자동 전환, 별도 개입 불필요
Notion 기반 무코드 설정 AI 지침·직원·프로젝트 정보를 Notion에서 직접 수정. n8n 코드 변경 없이 운영 가능
DB 이름 기반 동적 연결 Notion DB ID 하드코딩 배제. DB 재생성 시에도 이름으로 자동 재연결
Notion 기반 AI 응답 챗봇 서비스 Notion 페이지에 시스템프롬프트·추가데이터(회사 규정, 운영 지침 등)를
토글 블록 구조로 작성하면, Leave Notice 외 일반 메시지에 대해 Claude Haiku 4.5가 대화 메모리(Simple
Memory)를 유지하며 채널 내에서 직접 AI 응답. AI봇응답서비스 플래그로 on/off 제어 가능.
역등성 보장 사내메신저 edited_message 감지 → 기존 로그 삭제 후 재등록 자동 처리

업무 보고서 자동화

보고서	실행 시점	집계 범위
주간업무보고	매주 일요일 01:00	해당 주 월~일
월간업무보고	매월 1일 01:00	전월 1일~말일

Notion DB텔레그램로그 조회 → AI 마크다운 보고서 초안 생성 → Notion 페이지 저장

리소스 산정 공식

직급 가중치(직급별)와 시간대 가중치(평일주간/야간/휴일/휴가중)를 조합하여 실제 업무 부하를 정량화.

$$\text{최종산정리소스} = (1 + \text{직급가중치} + \text{시간대가중치}) \times \text{기본인정시간(h)}$$

AI 응답 형식

Leave Notice

1) 구분 : 외근

2) 일시 : 2026.06.26(금) 10:00~17:00

3) 인원 : 홍길동

☑ AI 분석

☑ 등록 완료 / P001 고속철도 / 외근

모든 직원 등록됨

포트폴리오: 사내메신저 + n8n + Notion 기반 근태관리 자동화 시스템

프로젝트 개요

사내메신저 봇 명령어로 직원 근태를 기록하고, 이를 Notion 데이터베이스에 자동 저장·조회·수정할 수 있도록 구축한 업무 자동화 시스템이다.

단순한 봇 연동을 넘어서, self-hosted n8n 환경과 외부 webhook 수신 구조, 데이터 중심 운영 설계, 예외 처리 흐름까지 포함한 실운영형 자동화 시스템으로 설계했다.

데모 영상

- 데모 영상: <https://youtu.be/eV88hV7wLAg>



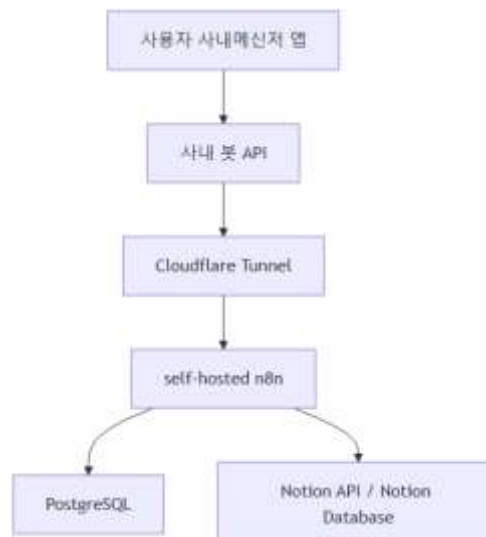
이 프로젝트의 목표는 다음과 같았다.

- 반복적인 근태 입력 과정을 메신저 명령어 기반으로 단순화
- 운영자가 코드 수정 없이 명령어와 설정을 변경할 수 있도록 데이터 중심 구조 설계
- 외부 접속 제약, 한글 컬럼 처리, 플랫폼 제한사항 등 실제 운영 환경의 문제를 안정적으로 해결

핵심 성과

- 사내메신저 명령어만으로 출근, 퇴근, 외근, 복귀, 조회, 삭제, 수정까지 처리하는 자동화 흐름 구현
- 명령어, 사용자 설정, 프로젝트 정보를 데이터베이스로 분리해 운영 중 무코드 변경 가능 구조 설계
- 외부 webhook 연결 제약이 있는 환경에서도 안정적으로 이벤트를 수신하는 self-hosted 자동화 환경 구성
- n8n 기본 노드 한계를 JavaScript Code 노드로 보완해 실사용 가능한 사용자 경험 구현
- 다수 사용자 알림 발송을 병렬 처리해 순차 방식 대비 처리 성능 개선

시스템 구성



구성 요소

구성 요소	역할
n8n	자동화 워크플로우 실행 엔진
PostgreSQL	워크플로우 및 설정 영속 저장
Cloudflare Tunnel	외부 webhook 수신 경로 안정화
Notion	근태, 명령어, 프로젝트 데이터 저장소
사내 봇	사용자 입력 및 결과 확인 인터페이스

주요 기능

1. 근태 기록 자동화

사내메신저 명령어를 통해 출근, 퇴근, 외근, 복귀를 기록할 수 있도록 구성했다.

- 당일 기록 조회
- 기록 삭제
- 프로젝트 지정 외근 등록
- 기존 기록의 프로젝트 정보 수정
- 상황에 따라 중복 허용, 갱신, 차단 정책이 다른 명령어 구조 적용

2. 데이터 중심 명령어 관리

명령어 정의를 코드에 하드코딩하지 않고 별도 데이터베이스로 분리했다.
이를 통해 운영자는 데이터만 수정해도 새 명령어를 추가하거나 설명을 바꿀 수 있고, 봇 메뉴도 자동 동기화되도록 구성했다.

3. 사용자별 근태 분리 처리

조회와 수정 흐름 전반에 사용자 식별 기준을 일관되게 적용해, 사용자별 기록이 섞이지 않도록 설계했다.
또한 알림 발송에 필요한 사용자별 설정 정보를 분리 관리해, 개인별 출근 기준 시각에 맞는 안내가 가능하도록 구성했다.

4. 외근/복귀 흐름 자동화

복귀 입력 시 당일 외근 기록 중 아직 짝이 맞지 않은 최근 항목을 우선 탐지해 자동 연결하도록 구현했다.
자동 탐지에 실패하면 수동 선택 흐름으로 자연스럽게 전환되도록 설계해, 자동화와 안정성을 함께 확보했다.

기술적으로 해결한 문제

1. 외부 webhook 수신 제약 해결

일반적인 공개형 환경이 아닌 self-hosted 환경에서 외부 메신저 플랫폼의 webhook을 안정적으로 수신해야 했다.
이를 위해 Cloudflare Tunnel 기반 연결 구조를 도입해, 직접 공개가 어려운 환경에서도 안정적으로 이벤트를 수신할 수 있도록 구성했다.

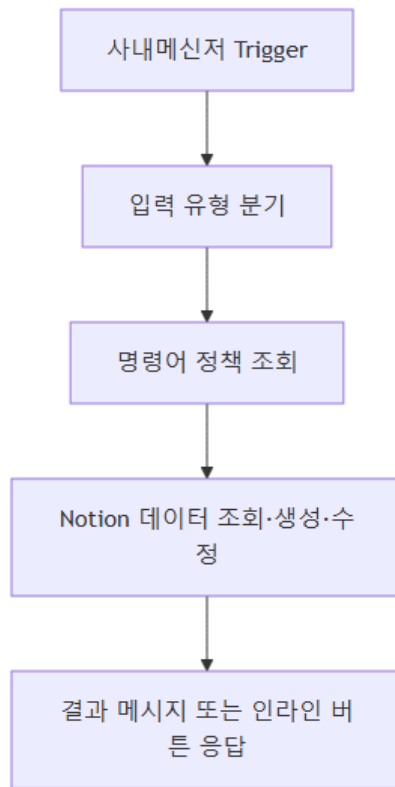
2. 사내 봇 callback 데이터 길이 제한 대응

인라인 버튼 기반 수정 기능을 구현하는 과정에서 플랫폼의 callback_data 길이 제한을 고려해야 했다.
식별자를 압축해 전달하고, 서버 측에서 다시 복원하는 방식으로 사용자 경험을 유지하면서 제약을 해결했다.
예시 포맷은 ep|{recordIdCompressed}|{projectIdPrefix} 형태로 설계했다.

대표 워크플로우

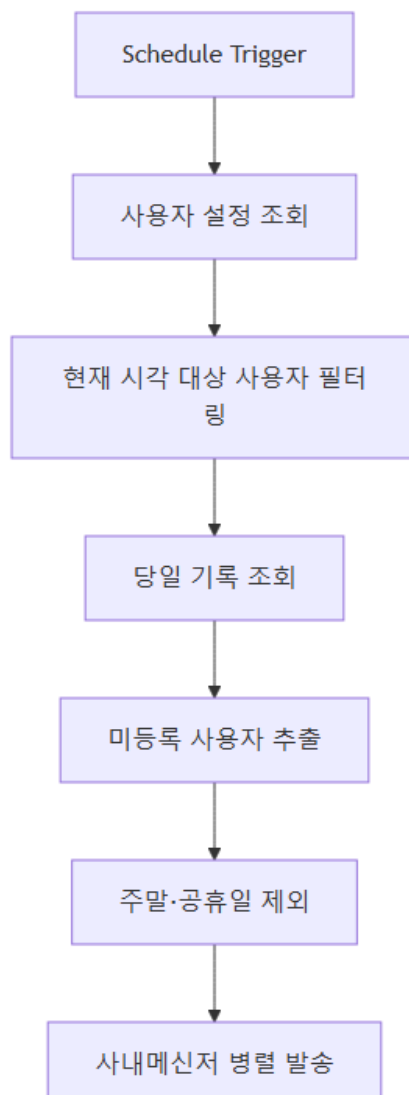
근태관리 워크플로우

핵심 워크플로우로, 사내메신저 메시지와 버튼 입력을 받아 근태를 기록·조회·수정한다.



출근 미등록 알림 워크플로우

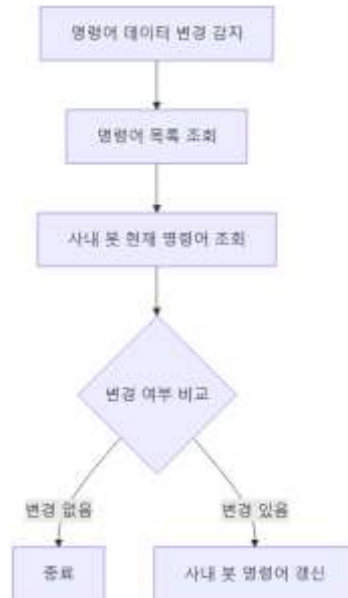
스케줄 기반으로 실행되며, 해당 시각에 출근해야 하는 사용자를 식별한 뒤 아직 기록이 없는 경우에만 알림을 발송한다.



- 사용자별 출근 기준 시각 반영
- 당일 기록 존재 여부 기반 알림 제외
- 공휴일 및 주말 예외 처리
- 병렬 발송을 통한 처리 시간 단축

명령어 동기화 워크플로우

명령어 데이터가 변경되면 사내 봇 명령 목록도 자동으로 동기화되도록 구성했다.



이를 통해 운영자는 코드를 수정하지 않고도 봇 메뉴를 최신 상태로 유지할 수 있다.

데이터 설계 요약

세부 식별자나 내부 스키마보다, 운영 구조가 명확하게 분리되어 있다는 점이 핵심이다.

데이터 영역	역할
근태 기록	출근, 퇴근, 외근, 복귀 등 이벤트 저장
사용자 설정	알림 기준 시각, 기본 프로젝트, 사용자별 옵션 관리
명령어 관리	봇 명령어, 설명, 동작 정책 정의
프로젝트 정보	외근 및 업무 연계용 프로젝트 선택 정보 저장

이 구조 덕분에 자주 바뀌는 운영 규칙을 코드가 아니라 데이터에서 제어할 수 있었다.

설계상 강점

NoCode + Code의 균형

기본 흐름은 n8n 노드로 구성하되, 플랫폼 기본 기능만으로 부족한 부분은 JavaScript Code 노드로 보완했다. 덕분에 개발 속도와 유연성을 동시에 확보할 수 있었다.

운영 친화적 구조

명령어, 사용자 설정, 프로젝트 목록을 데이터로 분리해 운영 중 변경 비용을 낮췄다. 이는 자동화 시스템이 단발성 데모가 아니라 실제 운영 가능한 형태가 되도록 만드는 핵심 요소였다.

예외 상황을 고려한 UX

자동 선택 실패 시 수동 선택으로 전환하고, 빈 조회 결과나 예외 케이스를 명시적으로 처리해 사용자 경험이 끊기지 않도록 설계했다.

비용 효율적 운영

상용 SaaS 의존을 최소화하고, self-hosted 환경과 무료·저비용 서비스 조합으로 실용적인 운영 구성을 완성했다.

사용 기술

영역	기술
자동화 엔진	n8n
데이터 저장	Notion API
인터페이스	사내 봇 API
네트워크 구성	Cloudflare Tunnel (개발 환경) / n8n Webhook (운영 환경)
구현 언어	JavaScript, Python
운영 환경	Docker, Ubuntu (개발환경) / n8n cloud (운영환경)

한 줄 정리

이 프로젝트는 메신저 기반 업무 자동화를 실제 운영 가능한 수준으로 설계하고, 플랫폼 제약과 운영 이슈를 직접 해결해낸 사례다.
특히 n8n 같은 자동화 도구를 단순 연결 수준이 아니라, 데이터 설계, 운영 안정성, 예외 처리, 사용자 경험까지 고려한 시스템으로 확장했다는 점에서 의미가 있다.

FamilyDoctor 포트폴리오

1. 프로젝트 개요

항목	내용
프로젝트명	FamilyDoctor - AI 주치의
유형	개인 프로젝트
기간	2026년 3월
배포 URL	https://familydoctor-sc6ue6cf3q-uc.a.run.app
저장소	https://github.com/c0z0c/familydoctor

가족 구성원의 건강 히스토리를 누적 관리하고, Gemini LLM이 오케스트레이터로 대화 흐름을 조율하며 MedGemma 4B GGUF 모델이 로컬에서 직접 의료 조언을 생성하는 웹 애플리케이션.

2. 기술적 도전 과제와 해결

2.1 로컬 LLM(GGUF)과 클라우드 서비스의 통합

문제: Google Cloud Run은 stateless 컨테이너 환경으로 2.5GB GGUF 모델을 매번 로드해야 한다. 모델 파일을 이미지에 포함하면 빌드/배포 시간이 급증하고, 환경 변수나 DB 연결처럼 일반적인 "준비 완료" 판단이 불가능하다.

해결:



- MedGemmaService를 별도 자식 프로세스(multiprocessing.spawn)로 분리하여 FastAPI 메인 프로세스와 메모리 공간 격리
- 부모-자식 간 통신을 multiprocessing.Queue로 처리, 추론 요청/응답을 비동기로 위임
- prepare_async()를 asyncio.create_task()로 백그라운드 실행하여 서버는 즉시 기동

2.2 Cloud Run cold start에서의 503 문제

문제: Cloud Run 기본 TCP probe는 uvicorn 포트가 열리면 즉시 Ready로 판단한다. GGUF 모델 예열(~41초)이 완료되기 전에 트래픽이 유입되어 /api/medical/chat이 503을 반환하는 상황이 반복됐다.

실측 데이터 (Cloud Run 로그, revision familydoctor-00014-wg7):

Cold Start 타임라인 (총 ~41초)



단계	시각	경과
워커 프로세스 기동 + 예열 대기 시작	08:22:39	0초
llama_context 출력 (모델 파일 읽기 완료)	08:23:16	+36.7초 (90%)
예열 추론 완료 (16토큰)	08:23:20	+40.6초 합계
/health 200 (startup probe 통과)	08:23:20	워커 완료와 동시
Cloud Run revision Ready	08:23:20	컨테이너 시작부터 1분 12초

해결:

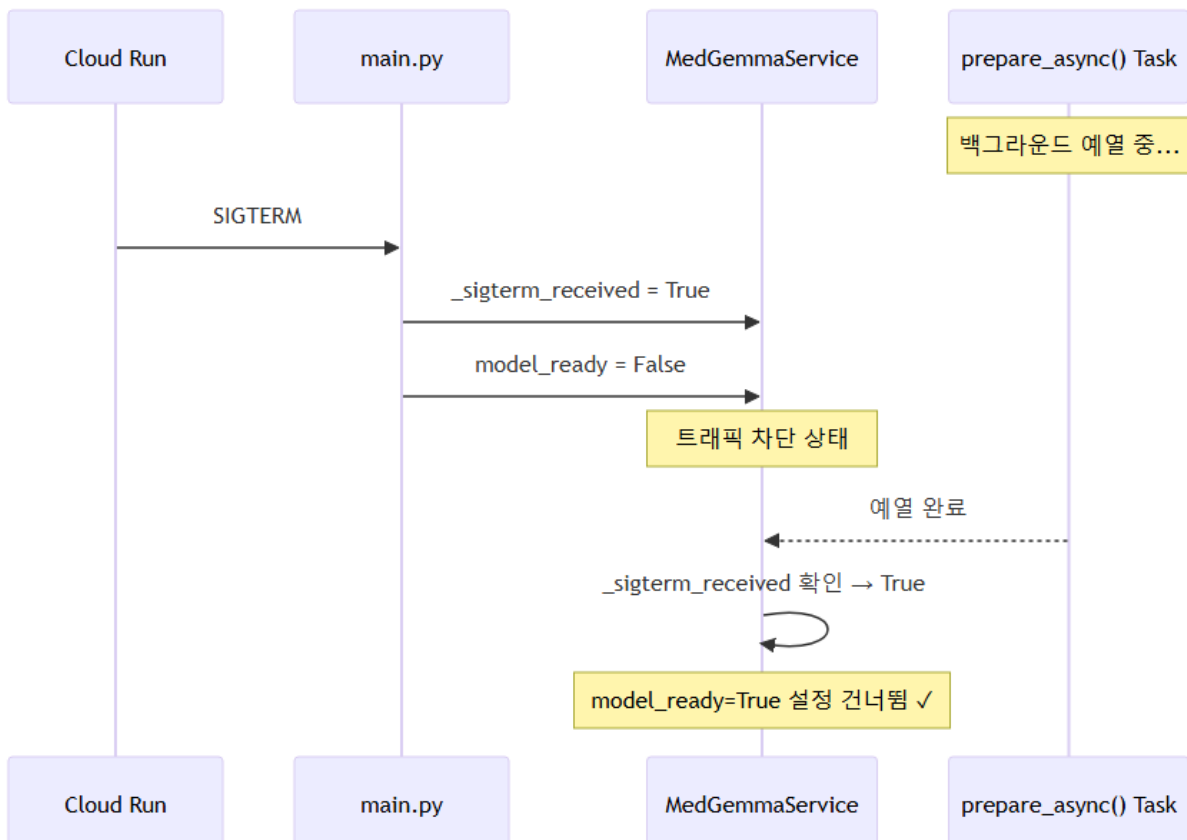
1. /health 엔드포인트가 model_ready=False일 때 HTTP 503 반환
2. cloudbuild.yaml에 startup probe 추가

```
--startup-probe=httpGet.path=/health,httpGet.port=8080,
initialDelaySeconds=10,periodSeconds=10,
failureThreshold=90,timeoutSeconds=5
```

1. startup probe가 /health 200을 확인한 후에만 트래픽 전환 → 503 완전 차단

2.3 SIGTERM 경합 조건 버그

문제: Cloud Run이 인스턴스 교체 시 SIGTERM을 보내면 _on_sigterm 핸들러가 model_ready=False로 설정한다. 그러나 백그라운드 예열 Task가 이후에 완료되면서 model_ready=True로 되돌려 SIGTERM을 받은 인스턴스가 다시 트래픽을 받는 문제가 발생했다.



해결:

```
# MedGemmaService
self.sigterm_received: bool = False

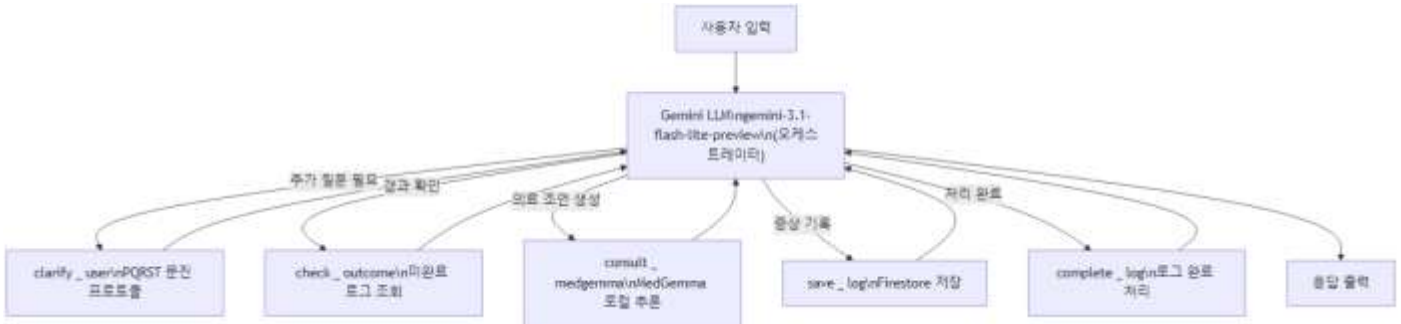
# main.py _on_sigterm 핸들러
medgemma._sigterm_received = True
medgemma.model_ready = False

# prepare_async() 완료 직전 가드
if self._sigterm_received:
    logger.warning("SIGTERM 수신 상태 - model_ready=True 설정 건너뛴")
    return
self.model_ready = True
```

2.4 LangGraph ReAct Agent Tool 설계

문제: 단순 Gemini API 직접 호출(GeminiService 레거시)로는 대화 흐름 제어, 로그 저장, MedGemma 호출을 하나의 요청 안에서 일관성 있게 처리하기 어려웠다.

해결: LangGraph create_react_agent로 전환하여 5개 Tool을 클로저 바인딩:



Tool	역할
clarify_user	PQRST 문진 프로토콜 기반 추가 질문
check_outcome	미완료 로그 경과 확인
consult_medgemma	MedGemmaService 로컬 추론 호출
save_log	증상 로그 Firestore 저장
complete_log	미완료 로그 완료 처리

Gemini LLM(gemini-3.1-flash-lite-preview)이 PQRST 문진 가이드와 중복 질문 금지 규칙이 포함된 시스템 프롬프트를 기반으로 Tool을 자율 선택한다.

2.5 비용 최소화 설계

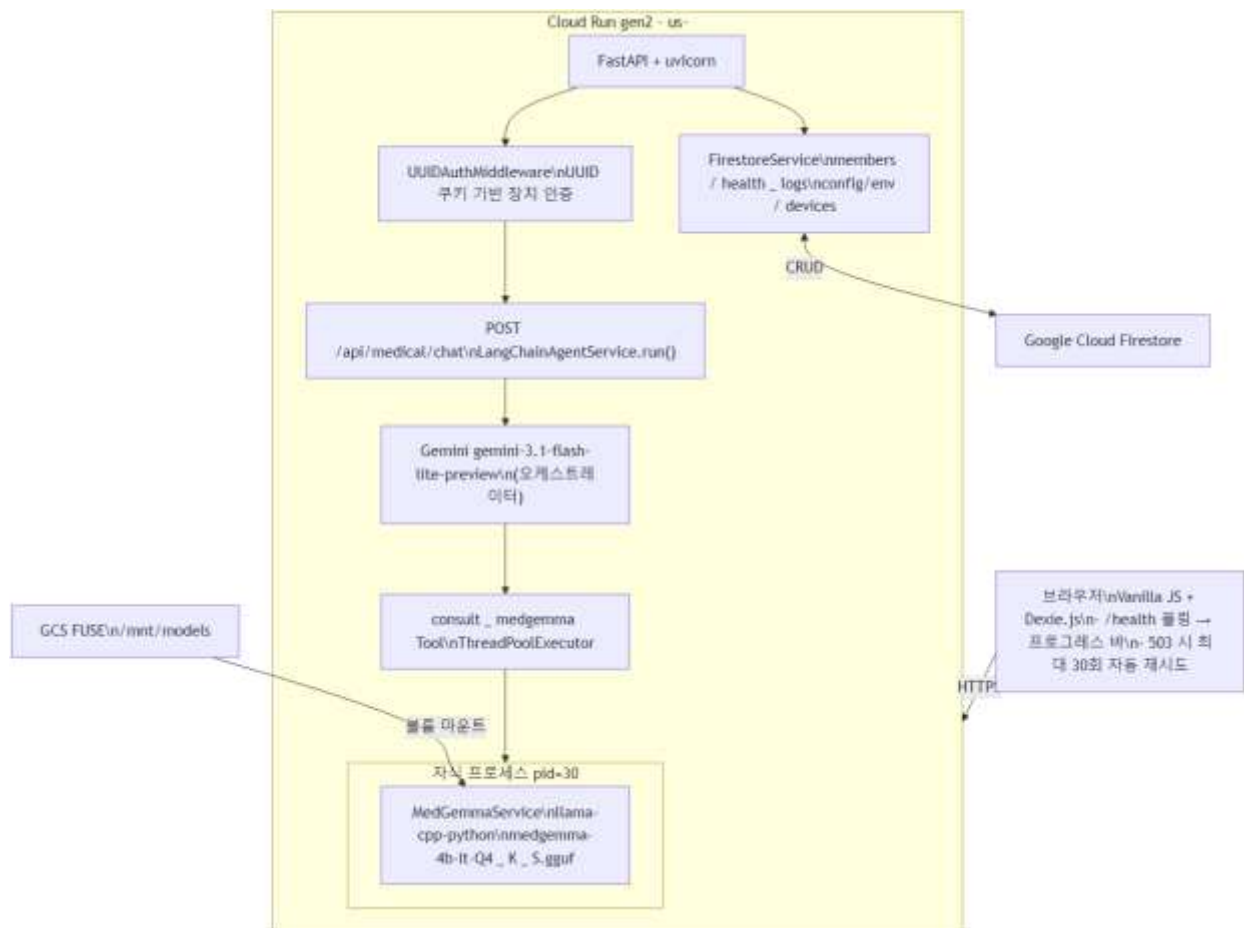
월 1~2회 호출하는 개인 프로젝트 특성상 비용 최소화가 핵심 요구사항이었다.

설정	이전	이후	이유
min-instances	0	0	idle 비용 없음 유지
max-instances	3	1	동시 사용자 없음
concurrency	10	1	단일 사용자
cpu-boost	활성	비활성	cold start 중 CPU 2배 과금 제거

예상 월 비용:

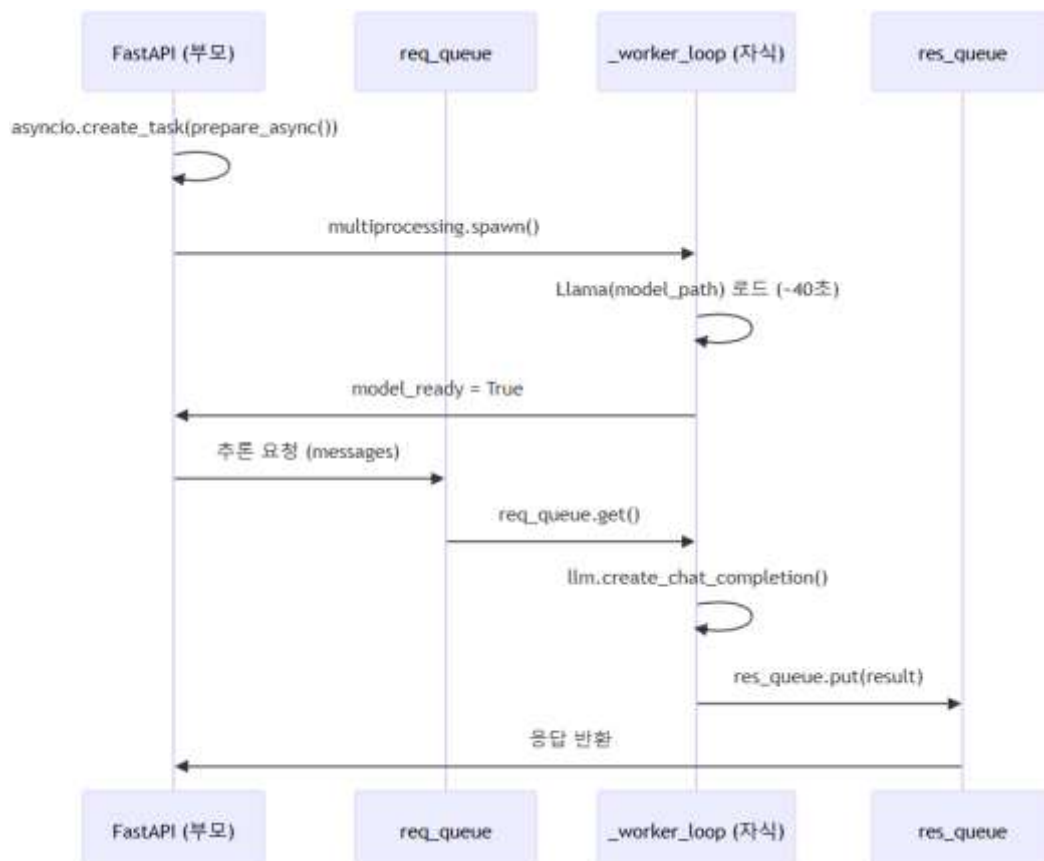
0.10 미만 (GCS 스토리지0.05 포함)

3. 시스템 아키텍처



4. 핵심 구현 코드

4.1 MedGemmaService 멀티프로세스 구조



```
# 자식 프로세스에서 모델 1회 로드 후 Queue로 요청 수신
def _worker_loop(req_queue, res_queue, model_path):
    llm = Llama(model_path=model_path, n_ctx=4096, n_threads=1)
    while True:
        req = req_queue.get()
        if req is None: # 종료 신호
            break
        raw = llm.create_chat_completion(messages=req["messages"], ...)
        res_queue.put({"ok": True, "result": raw["choices"][0]["message"]["content"]})

# 부모 프로세스(FastAPI)에서 비동기 위임
async def prepare_async(self):
    await loop.run_in_executor(self._executor, self._ensure_worker)
    await loop.run_in_executor(self._executor, self._warmup) # 실효 40.6초
    if not self._sigterm_received:
        self.model_ready = True
```

4.2 /health startup probe 연동

```
@app.get("/health")
async def health_check(request: Request):
    medgemma = request.app.state.medgemma
    body = {
        "status": "healthy" if medgemma.model_ready else "initializing",
        "model_ready": medgemma.model_ready,
        "model_downloading": medgemma.model_downloading,
    }
    if not medgemma.model_ready:
        return JSONResponse(content=body, status_code=503)
    return body
```

4.3 프론트엔드 서버 로딩 폴링

```
async function waitForServer() {
    if (await checkServerReady()) return; // 이미 warm이면 즉시 통과

    overlay.classList.remove("hidden"); // 프로그레스 바 오버레이 표시
    // 1초마다 경과 시간 갱신, 10초마다 /health 폴링
    // model_ready=true 되면 오버레이 자동 제거
}
```

5. 개발 회고

잘 된 점

- GCS FUSE 볼륨 마운트로 이미지 크기와 모델 파일 관리 분리 성공
- startup probe 도입으로 cold start 중 503 문제 완전 해결
- SIGTERM 경합 조건을 _sigterm_received 플래그로 명확하게 가드
- LangGraph ReAct Agent 전환으로 대화 흐름 제어 코드가 선언적으로 단순화

개선 여지

- GCS FUSE 네트워크 읽기로 인한 cold start 5~6분은 모델 파일을 Docker 이미지에 포함하면 30~60초로 단축 가능 (Artifact Registry 비용 증가 트레이드오프)
- n_ctx=4096으로 제한 중 (n_ctx_train=131072). 메모리 확보 시 확장 가능
- n_threads=1로 단일 스레드 추론 중. CPU 2개 활용을 위해 조정 가능
- 개인 프로젝트로 비용은 최소화 하였으나 더 빠른 응답이 필요하면 항상 켜진 인스턴스(min-instances=1)로 변경 고려 가능

6. 기술 스택 요약

Language : Python 3.12
Framework : FastAPI, LangChain, LangGraph
LLM : Google Gemini gemini-3.1-flash-lite-preview (오케스트레이터)
MedGemma 4B GGUF Q4_K_S (로컬 추론, llama-cpp-python)
Database : Google Cloud Firestore
Frontend : Vanilla JS, Dexie.js, marked.js
Infra : Google Cloud Run gen2, GCS FUSE, Cloud Build, Artifact Registry
Auth : UUID 기반 장치 인증 (Firestore devices 컬렉션)



미션: 소음 속에서도 즉각 반응하는 데모 환경 구축



The Goal

고성능 지향성 마이크의 소음 차단(Noise Canceling) 및 수음 능력을 직관적으로 시연.

The Constraint

현장 소음 환경에서도 AI가 사용자의 의도를 정확히 파악해야 함.



Key Objectives

- High-Responsiveness:** '자연스러운 대화 속도'를 위한 지연시간(Latency) 최소화.
- Zero-Touch UI:** 버튼 조작 없이 목소리만으로 주문부터 결제까지 완결.
- Make Hardware Shine:** 소프트웨어적 기교보다는 하드웨어의 입력 성능을 투명하게 보여주는 역할.



보이지 않는 인터페이스 (Invisible Interface)

• Hands-Free Interaction

고성능 VAD(Voice Activity Detection)를 적용하여 버튼 클릭 없는 자연스러운 발화 감지.

• Real-time Visualizer

마이크 입력 신호를 실시간 파형으로 시각화하여 하드웨어 동작 여부를 직관적으로 피드백.

• Live Chat UI

음성 인식 결과(STT)와 AI 응답을 채팅 형태로 보여주어 인식 정확도를 즉시 검증.

초저지연 AI 대화 엔진 (Core Engine)



Multilingual Support

한국어/일본어 실시간 전환 및 각 언어별 전용 페르소나(Assistant ID) 매핑.

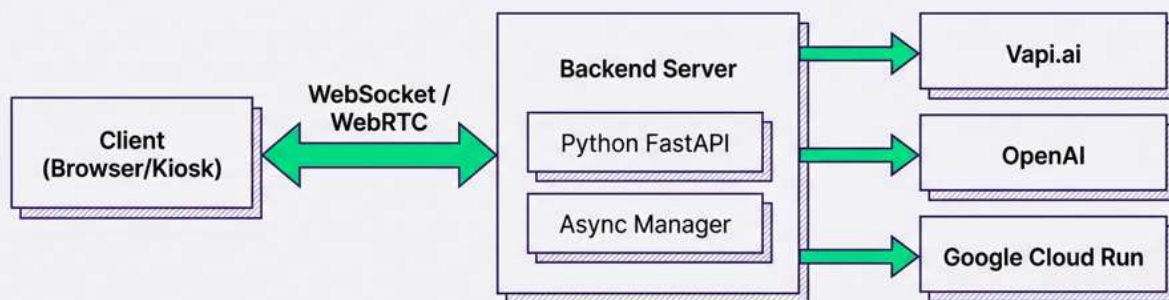
Scenario Logic

주문 → 확인 → 가상 결제 → 호출로 이어지는 완결된 프랜차이즈 시나리오 구현.

Fallback Strategy

Vapi.ai 미들웨어와 OpenAI 직접 연결 방식을 상황에 따라 선택 가능한 유연한 설계.

견고한 백엔드 아키텍처 (System Architecture)



- **Server Framework:** Python FastAPI (Async 비동기 처리 지원).
- **Communication:** WebSocket을 이용한 양방향 실시간 데이터 스트리밍.
- **Data Flow:** 오디오 스트림과 제어 신호(Control Signals)의 효율적인 라우팅.
- **Dependency Injection:** 기기 인증 및 DB 연결을 위한 모듈화된 의존성 주입 패턴 사용.

중앙 집중식 환경 변수 관리 시스템

Problem: 데모 현장에서 API 키나 프롬프트 변경 시 서버 재배포 불가능.

Solution: Firestore 기반의 실시간 Config 관리 대시보드 구축.

- **Hot-Swapping:** 서버 재시작 없이 API Key, Assistant ID, Test Mode 등 핵심 설정 즉시 변경.
- **Security:** SHA-256 해시 기반의 관리자 비밀번호 인증.
- **Visual Editor:** JSON 형식이 아닌 직관적인 Web UI 제공.

환경 변수 설정

관리자 비밀번호 변경

AMPLIFICATION	4
AUTH_UUID_ENABLE	false
HOST	0.0.0.0
MENU_INFO_ENABLED	false
PORT	8080
RECOGNITION_TEXT_ENABLED	false
TEST_MODE	false
USE_FIRESTORE	true
VAPI_ASSISTANT_ID	a248b6a0-47ad-43b1-9c4d-2976a5b50de4
VAPI_ASSISTANT_KR_ID	d1ddfc8f-4f26-4490-94fa-cdd9bab8f1c
VAPI_KEY	5179a2d3-aa8a-48d6-88bc-85b643a1d1e0

저장

기기 인증 및 접근 제어 (Security & Access Control)

UUID Whitelist

브라우저 로컬 스토리지에 생성된 UUID를 기반으로 승인된 기기만 API 호출 허용.

Expiration Control

만료일(Expires_at) 설정을 통해 임시 데모용 기기의 접근 권한 자동 회수.

기기 인증 관리

한줄 상태: X AUTH_UUID_ENABLE 비활성화됨 (모든 기기 접근 허용)

기기 ID	상태	생성일시	최근 접속	만료일	비고	작업
339d8017-df1a-4f8f-b78e-8af547fad0d7	승인됨	2026. 02. 07. 오후 12:47	2026. 02. 07. 오후 01:35	-	평판계열 PC(google)	수정 삭제
bc731960-9660-4270-b137-2045d6c81b89	승인됨	2026. 02. 06. 오후 03:58	2026. 02. 07. 오후 12:20	-	평판계열 PC(local)	수정 삭제
c88e7a1b-96aa-4d2a-a0df-7fe016172386	승인됨	2026. 02. 07. 오후 01:17	2026. 02. 07. 오후 01:43	-	평판계열 PC(local)	수정 삭제

Lifecycle Management

승인(Approved), 대기(Waiting), 거부(Rejected) 상태 관리.

Cost Management

무분별한 토큰 사용 방지를 위한 엄격한 세션 제어.

어디서든 실행 가능한 배포 환경 (DevOps)



Containerization

Dockerfile을 통한 환경 표준화 (개발, 테스트, 운영 환경 일치).

Cloud & Local

Google Cloud Run을 통한 서버리스 배포 및 로컬 노트북 단독 구동 모두 지원.

Stability

WebRTC 및 WebSocket 연결 유지를 위한 네트워크 최적화 설정.

프로젝트 성과 및 핵심 가치



Performance

하드웨어(마이크) 성능 검증을 위한 신뢰성 높은 테스트 베드 역할 수행.



Completeness

음성 인식부터 관리자 대시보드까지 Full-Cycle 자체 개발.



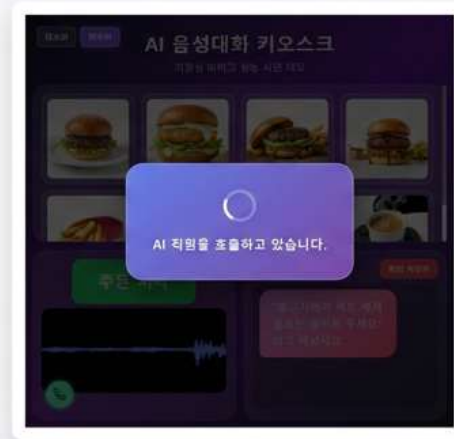
Reliability

다국어(KR/JP) 환경에서의 안정적인 WebSocket 연결 및 예외 처리.



Documented

Swagger/ReDoc을 통한 API 문서화 및 상세 개발 보고서 작성.



Hyper-Personal AI 비서 구축 (OpenClaw & Gemini 3.1)

"이론을 넘어 실행으로 증명하는 AI 컨설팅" 하남시 개인 서버 환경에 오픈소스 MCP 게이트웨이(OpenClaw)와 최신 LLM(Gemini 3.1)을 통합하여, 개인 업무 생산성을 극대화하고 전사 AI 도입의 프로토타입을 제시한 프로젝트입니다.

1. 프로젝트 개요 (Executive Summary)

- **배경:** 단순 챗봇을 넘어, 개인의 실제 업무 데이터(Notion) 및 메신저(Telegram)와 연동되어 '행동'할 수 있는 진정한 의미의 AI 비서 필요성 대두.
- **목표:**
 - a. 오픈소스 MCP(Model Context Protocol) 게이트웨이인 OpenClaw의 안정적 인프라 구축.
 - b. 2026년 최신 모델인 Gemini 3.1 Flash Lite의 강력한 컨텍스트 처리 능력 활용.
 - c. 텔레그램을 통한 접근성 및 노션(Notion) 기반의 자동 보고/기록 시스템 설계.
- **구현 범위:** 텔레그램 봇 + OpenClaw + Gemini 3.1 연동 프로토타입 동작 확인. 노션 연동은 아키텍처 설계 완료 (별도 n8n 프로젝트에서 노션 API 연동 검증).
- **성과:** 온프레미스 환경에서 MCP 게이트웨이 + LLM 통합 아키텍처 실증. 사내 AI 컨설팅 도입을 위한 기술 기반 마련.

2. 핵심 기능 (Key Features)

🗣️ Gemini 3.1 기반의 지능형 대화

- 최신 모델인 gemini-3.1-flash-lite-preview를 탑재하여 응답 속도와 정확도 확보.
- 긴 대화 맥락(Context Window)을 유지하며 복잡한 업무 지시 수행 가능.

📁 텔레그램-노션 자동 보고 시스템 (설계 완료 / 미구현)

- 텔레그램 대화 내용을 AI가 분석하여 노션 데이터베이스에 업무 로그, 태스크, 일정을 자동 기록하는 구조로 설계.
- 노션 API 연동 자체는 별도 n8n 프로젝트에서 검증 완료. OpenClaw 프로토타입에서는 미적용.

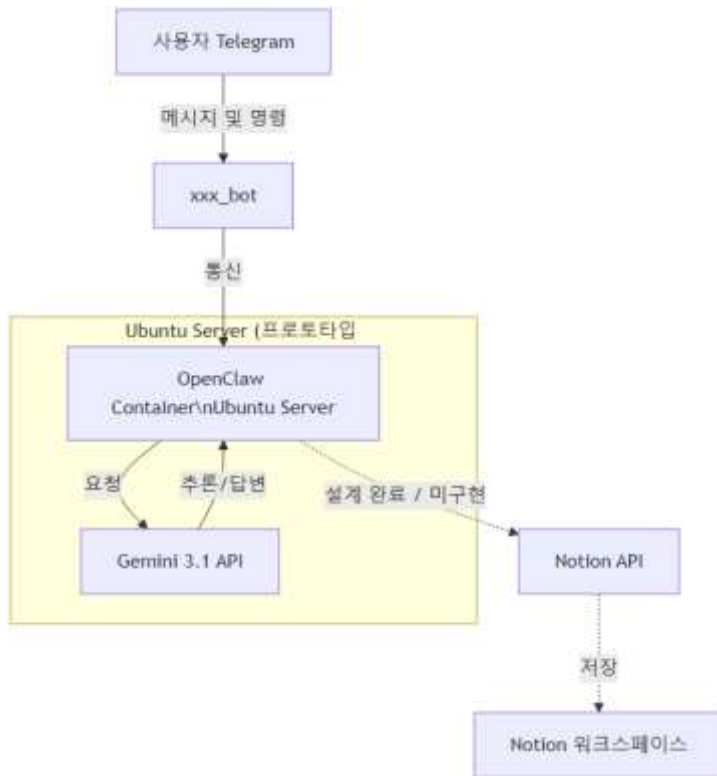
🔧 Docker 기반의 안정적인 온프레미스 구축

- 개인 Ubuntu 서버에 Docker Container 환경으로 구축하여 Host 시스템과 격리된 안전한 운영.
- 호스트 네트워크 모드 활용으로 향후 사내 인프라 점검 도구(커스텀 MCP) 확장을 위한 토대 마련.

3. 기술 스택 및 아키텍처 (Technical Stack)

구분	기술 요소	역할
Infrastructure	Ubuntu 24.04 LTS (Headless)	호스트 OS (15Gi RAM 환경)
Container	Docker & Docker Compose	애플리케이션 가상화 및 배포 관리
AI Gateway	OpenClaw (MCP Gateway)	LLM과 외부 인프라(메신저, DB) 연결
LLM	Google Gemini 3.1 Flash Lite	메인 AI 두뇌 (대화 및 로직 처리)
Interface	Telegram Bot API	사용자 인터페이스 (모바일/PC 어디서나 접근 가능)
Database/Memory	Notion API	업무 로그 저장 및 장기 기억 장치 (설계 / n8n에서 검증)

🔗 System Architecture Diagram



실선: 프로토타입에서 구현 완료 / 점선: 설계 완료, 미구현 (n8n 프로젝트에서 Notion API 연동 별도 검증)

CNN 기반 문서 이미지 복원 프로젝트

핵심 성과: PSNR 35.21dB 달성 | U-Net + Triple Composite Loss | 기본 모델 대비 +15.7%

GitHub 소스코드 Colab 실행하기

프로젝트 개요

항목	내용
기간	2025년 8월
역할	모델 설계 / 구현 (단독)
핵심 기술	PyTorch, U-Net, VGG16 Perceptual Loss, CosineAnnealingWarmRestarts
데이터	Kaggle Denoising Dirty Documents (144개 → 864개 증강)

손상된 문서 이미지에서 노이즈를 제거하여 원본을 복원하는 딥러닝 모델을 개발했습니다. 문서 이미지의 흑백 쌍봉 분포(Bimodal Distribution) 특성을 분석하고, 이에 최적화된 손실 함수 조합을 설계하여 기본 모델 대비 PSNR +15.7% 향상을 달성했습니다.

기술적 도전과 해결

과제	해결 방안
흑백 쌍봉 분포	L1 + SSIM + Perceptual 복합 손실(Triple Composite Loss)로 간접 달성
텍스트 경계 번짐	SSIM Loss로 구조적 유사성 강화
소규모 데이터셋(144개)	6배 데이터 증강

Triple Composite Loss

$$\mathcal{L}_{total} = 0.6 \cdot \mathcal{L}_{L1} + 0.2 \cdot \mathcal{L}_{SSIM} + 0.2 \cdot \mathcal{L}_{Perceptual}$$

손실 함수	역할
L1 Loss	픽셀 단위 정확한 위치 학습
SSIM Loss	구조적 유사성 및 경계선 보존
Perceptual Loss	VGG16 고수준 특징으로 대비 강화

모델 비교 실험

5개 모델을 설계·비교하여 최적 구성을 도출했습니다.

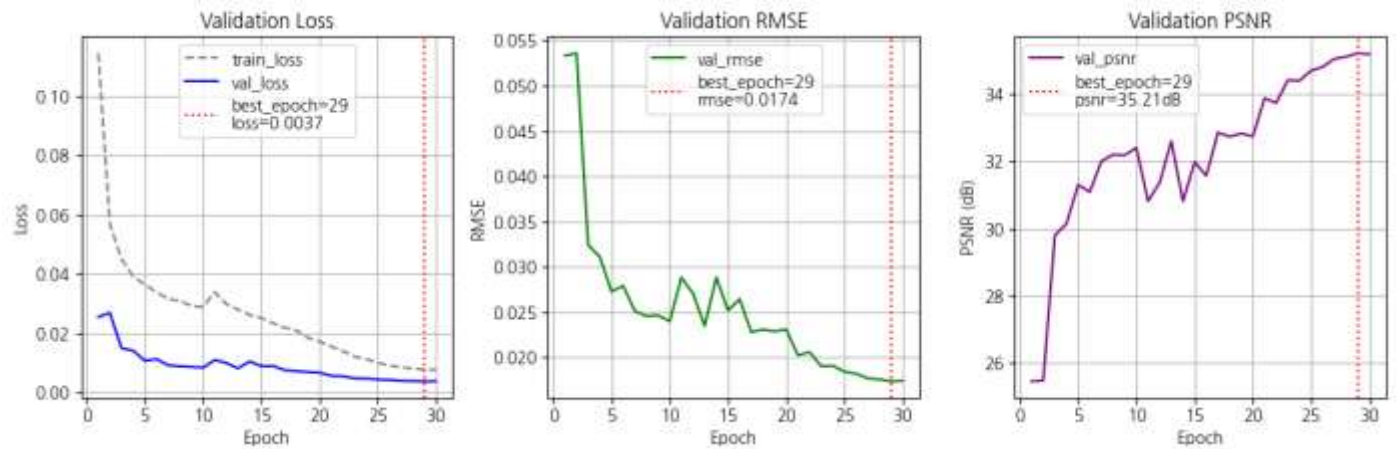
모델	아키텍처	손실 함수	PSNR (dB)
U-Net v2	U-Net v2	Triple Composite	35.21
U-Net v3	U-Net v3 (얇음)	SSIM	33.16
Attention ResU-Net	Attention ResU-Net	Triple Composite	32.53
AutoEncoder	AutoEncoder	L1	30.44
U-Net 표준	U-Net 표준	Triple Composite	26.03

U-Net v2 개선 포인트:

항목	v1 → v2 변경	효과
Dropout	제거	소규모 데이터에서 정보 손실 방지
가중치 초기화	He(Kaiming) 적용	수렴 안정성 향상
학습률 스케줄러	CosineAnnealingWarmRestarts	로컬 최솟값 탈출

결과 시각화

학습 곡선



복원 결과

검증 데이터:

입력(노이즈)	복원(모델)	정답(라벨)
There are several classic spatial filters for reducing frequency noise from images. The mean filter, the median filter, and the opening filter are frequently used. The mean filter is a filter that replaces the pixel values with the neighborhood average. The median filter replaces the pixel value with the median of the pixel neighborhood for each pixel, thereby reducing salt and pepper noise. Finally, the opening closing filter is a mathematical morphology operation that combines the same number of erosion and dilation morphological operations in order to eliminate small objects from images. The main goal was to train a neural network in a supervised learning manner to obtain a clean image from a noisy one. In this particular case, we obtain a simulated noisy image from a clean one through a noise addition process. The process for obtaining simulated noisy images is as follows: This process requires images of the background and the acquisition forms, which were obtained by printing	There are several classic spatial filters for reducing frequency noise from images. The mean filter, the median filter, and the opening filter are frequently used. The mean filter is a filter that replaces the pixel values with the neighborhood average. The median filter replaces the pixel value with the median of the pixel neighborhood for each pixel, thereby reducing salt and pepper noise. Finally, the opening closing filter is a mathematical morphology operation that combines the same number of erosion and dilation morphological operations in order to eliminate small objects from images. The main goal was to train a neural network in a supervised learning manner to obtain a clean image from a noisy one. In this particular case, we obtain a simulated noisy image from a clean one through a noise addition process. The process for obtaining simulated noisy images is as follows: This process requires images of the background and the acquisition forms, which were obtained by printing	There are several classic spatial filters for reducing frequency noise from images. The mean filter, the median filter, and the opening filter are frequently used. The mean filter is a filter that replaces the pixel values with the neighborhood average. The median filter replaces the pixel value with the median of the pixel neighborhood for each pixel, thereby reducing salt and pepper noise. Finally, the opening closing filter is a mathematical morphology operation that combines the same number of erosion and dilation morphological operations in order to eliminate small objects from images. The main goal was to train a neural network in a supervised learning manner to obtain a clean image from a noisy one. In this particular case, we obtain a simulated noisy image from a clean one through a noise addition process. The process for obtaining simulated noisy images is as follows: This process requires images of the background and the acquisition forms, which were obtained by printing

테스트 데이터:

입력(노이즈)	복원(모델)
A new offline handwritten database for the Spanish language has recently been developed: the Spartacus database (Spanish Restricted-domain Task of Cursive Script). There were two main goals in developing this corpus. First of all, most databases do not contain Spanish. Spanish is a widespread major language. Another important reason for developing this corpus is that these tasks are commonly used in the use of linguistic knowledge beyond the lexicon level in the recognition of handwritten text. As the Spartacus database consisted mainly of short sentences and paragraphs, the writers were asked to copy a set of sentences in line fields in the forms. Next figure shows one of the forms used. These forms also contain a brief set of instructions given to the	A new offline handwritten database for the Spanish language has recently been developed: the Spartacus database (Spanish Restricted-domain Task of Cursive Script). There were two main goals in developing this corpus. First of all, most databases do not contain Spanish. Spanish is a widespread major language. Another important reason for developing this corpus is that these tasks are commonly used in the use of linguistic knowledge beyond the lexicon level in the recognition of handwritten text. As the Spartacus database consisted mainly of short sentences and paragraphs, the writers were asked to copy a set of sentences in line fields in the forms. Next figure shows one of the forms used. These forms also contain a brief set of instructions given to the

회고

- 문서 이미지의 도메인 특성 분석(쌍봉 분포)이 손실 함수 설계의 핵심 근거가 됨
- Skip Connection 유무에 따른 성능 차이 실증 (U-Net v2 vs AutoEncoder: +4.77dB)
- 향후 GAN 기반 생성적 복원 또는 Vision Transformer 적용으로 추가 성능 개선 가능

프로젝트 기간: 2025년 8월

오픈소스 라이브러리 (PyPI)

"실무 문제를 패키지로 만들어 공개했습니다"

패키지	해결한 문제
helper-hwp	HWP 5.x·HWP97·HWPX 3종 포맷 파싱, CFB 바이너리 스펙 직접 분석·구현, AES 암호화 HWP 해제, 텍스트·MD·HTML·DOCX·PDF 다중 변환 + CLI 12개 명령
helper-md-doc	Markdown ↔ HTML ↔ DOCX ↔ PDF 양방향 변환, Mermaid·KaTeX 렌더링, D2Coding 한글 폰트 내장

- 논문 투고 진행중 (학회 블라인드 심사 규정 준수를 위해 제목을 마스킹 처리했습니다)
- 연구 내용: AI 빅데이터 학습 및 대용량 데이터 전처리 관련 연구



교육 과정 프로젝트 (Codeit AI 부트캠프, 2025.07 ~ 2026.01)

프로젝트	주요 기술
은행 정기예금 가입 예측 (분류)	XGBoost, Soft Voting 앙상블, ROC-AUC 0.8155
자전거 공유 수요 예측 (회귀)	ElasticNet + 로그 변환, R² 0.850
CNN 기반 문서 이미지 복원	PyTorch, U-Net, PSNR 35.21dB
SaaS 광고 콘텐츠 생성 플랫폼	FLUX.1-dev, MSA, FastAPI
RAG 나라장터 입찰 문서 검색	LangChain, FAISS, OpenAI

경력 요약

기간	소속	직위	주요 분야
2025.04 ~ 현재	프리랜서	AI 자동화 컨설턴트	LLM 자동화, n8n, MCP
2011.01 ~ 2025.05	(주)중앙첨단소재 연구소	수석연구원 / 팀장	철도 방송·보안, 전력 계량, 병상 방송
2002.01 ~ 2011.04	(주)셀런	차장 / 팀장	LG IPTV, VOD 플랫폼, 화상회의

학력 및 교육

기간	내용
2026.03 ~ 재학 중	서울사이버대학교 인공지능학과 3학년 편입
2025.07 ~ 2026.01	Codeit AI 엔지니어 부트캠프 수료 (총 1,250시간)
졸업	농협대학교 전자계산학과
2000.08 ~ 2000.10	필리핀 어학연수 (3개월)
병역	해병대 만기 전역